学习情境 L09：FAQ 直答与反馈闭环—让正确被复用（6 学时）

载体：DAP-v2 已投产的 FAQ 直答链路（同题命中直答 420ms，对比 LLM 全链路 56s，约 130 倍）。

主线案例：真实生产事故"FAQ 断路"（双层根因 =Milvus 索引元数据缺字段 × L2 检索透传丢失，健康路径恒不命中）。

一、学习情境基本信息

项目	内容
学习领域	LF3 智能查询服务开发：NL2SQL 与知识引擎（本域第 3 讲：检索 → 直答）
讲次	第 09 讲（第 9 周）
学时	6 学时 = 校内理实一体 4 学时（六步法 240'） + 企业实训站 2 学时（PR 实战）
布鲁姆主层级	**分析**（渗透：理解、应用、评价）
双元场地	校内：理实一体教室（演示区 + 4 人工位区）；企业站：10.167.2.175/176 双节点 K8S 集群 · dap-v2 课程分支 `edu/2026-spring`
企业站 PR 任务	以 analyst 角色提交一条真实反馈（corrected_sql 过四层护栏预校验），验证同题直答命中，附前后耗时对比证据
课后费曼任务	讲清"知识引擎是 Agent 效果天花板"——为什么模型再强，也救不了错误或丢失的知识

二、任务描述

【委托单 #R-2409】委托方：零售运营部王总监。上周例会"各品类 GMV"口径又答错一次，分析师手工改对了 SQL；可同样的问题下周还会再问，每次都要等大模型跑 56 秒、还可能再错。请把这次人工修正沉淀进知识库，让同题下次 1 秒内直接答对；同时保证坏 SQL 一行也进不了库。——数据平台组 DAP-v2 迭代周任务单

三、学习目标

1. 【理解】画出 FAQ 反馈闭环五环（feedback 端点 → few_shots 表 → Milvus 增量索引 → L2 检索透传 → 同题直答），说出每一环的输入、输出与失败形态。
2. 【应用】按操作单完成三组对照实验，实证闭环上的三个安全设计：RBAC（analyst+）、corrected_sql 四层护栏预校验、幂等 UPSERT。
3. 【分析】依据"few_shots 有行、同题恒走 56s 慢路径、且无任何报错"的现象，用接缝二分法（五问清单）定位"FAQ 断路"事故的双层根因：索引元数据缺字段 × L2 透传丢失。

4. 【分析】拆解 420ms 直答与 56s LLM 路径的耗时构成，量化知识复用在延迟与正确性两个维度的收益（约 130 倍 + 口径固化）。

5. 【评价】审查 57 条回归测试的覆盖逻辑，为链路再设计"第 58 条"断路测试，并说明其防回归价值。

四、教学重点与难点

教学重点

1. 反馈闭环全链路与三个安全设计（RBAC / 护栏预校验 / 幂等 UPSERT）的协作语义——"越权不进、先验后存、重放不重"。

2. "FAQ 断路"双层根因的分析方法与**断路测试思维**：集成链路每个接缝都要有测试证明"通"；单测全绿 ≠ 链路通。

教学难点（附突破策略）

1. **静默降级型故障的定位**：系统不出错、只变慢，无 error 日志可 grep，监控"全绿"。

- 突破策略：①计时基线法——同题两问用 `time curl` + 路由标记日志（route=faq/llm）把"慢"变成可观测证据；②接缝二分五问清单（存储在吗→索引进了吗→命中了吗→元数据全吗→透传到了吗），把开放问题拆成 5 个可独立验证的封闭问题，逐缝排除。

2. **幂等 UPSERT 与护栏预校验的顺序语义**：为什么必须先校验后落库、为什么重放两次只有一行。

- 突破策略：实证三步——重放两次比对响应 `dedup` 标记、前后 `SELECT` 行数与 version 变化、读 UPSERT 语句找 `ON CONFLICT` 去重键；再反例研讨"先存后验"会给 few_shots 带来什么脏数据（恶意 SQL 已落库、索引已污染）。

五、教学准备

环境（讲师课前 1 天核验；故障时按 RUNBOOK 降级为录屏 + 离线日志包）

- 双节点 K8S 集群（10.167.2.175/176）`dap` 命名空间：gateway / insight-agent / insight-worker / llm-gateway / crdb / milvus / redis 全部 Ready（`kubectl -n dap get pods`）。
- 课程分支 `edu/2026-spring` 就绪；每人发放三类课程 Token：viewer（只读）/ analyst（可反馈）/ owner（演示审批）。
- 课程租户 `tenant=edu`；学生可写数仓对象限 TiDB `edu_shadow` 库。

资料

- 代码：`dap-v2/services/insight/api/feedback.py`（端点）、`services/insight/guardrails/sql_check.py`（四层护栏）、`services/insight/faq/indexer.py`（增量索引 worker）、`services/insight/faq/retriever.py`（命中判定 + L2 透传）。
- 测试：`dap-v2/tests/faq/test_faq_regression.py`（57 条断路回归测试）。
- 事故档案：`docs/07-项目资源/incidents/INC-faq-break.md`（脱敏复盘报告）。
- 工具卡：`make gateway-forward` / `make crdb-sql` / `make insight-logs` 三个封装命令说明。

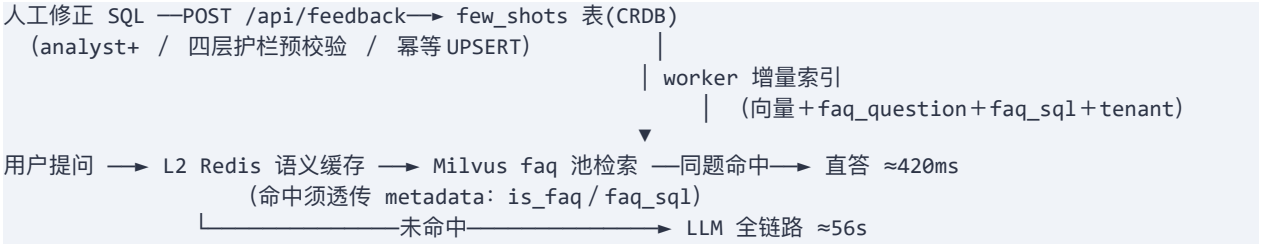
分组：4 人一组，角色轮换：操作员 / 计时记录员 / 日志观察员 / 评审员。

六、教学过程设计

第 1 步：资讯（30 分钟）

【教师活动】

- 双盲演示：同屏连问两次同一问题。第一次约 56s 出结果；第二次 0.4s 出同样结果。追问："第二次为什么快？快的那次真的对吗？如果每次都 56 秒，业务方会怎么用这个系统？"
- 板书反馈闭环五环图，每环标注一个"断点可能"：



- 发放引导文：委托单 #R-2409 + 五问清单 + 代码路径清单。

【学生活动】

- 观察两次请求响应差异（`route`、`elapsed_ms`、`faq_source` 字段）。
- 带着引导文的 3 个资讯问题通读 `feedback.py` 与 `indexer.py`：① corrected_sql 落库前经历了什么？② few_shots 到 Milvus 之间谁在搬数据、搬了哪些字段？③"同题"判定发生在哪一层？

【教学方法】演示法 + 引导文法（双元制资讯环节典型方法）。

【设计意图】用 130 倍时差制造认知冲击，把"知识复用"从抽象概念变成可测现象；引导文驱动学生读生产代码而非听讲。

第 2 步：计划（30 分钟）

【教师活动】给出任务分解模板与安全红线（只写 edu_shadow、只用自己的 tenant、幂等重放不超过 2 次、恶意 SQL 只用清单里的三条）。

【学生活动】小组产出实施计划表（子任务 / 关键命令 / 预期证据 / 风险），至少覆盖：

子任务	关键命令/路径	预期证据
直答 vs 慢答基线	`time curl` 同题两问	route=llm≈56s；route=faq≈0.42s
反馈三组对照	`POST /api/feedback`	200 + dedup / 403 / 422×3
增量索引观察	`make insight-logs f=faq_index`	4 个元数据字段齐全
断路事故复盘	五问清单逐缝验证	每问一条验证命令 + 结论

【教学方法】小组计划法 + 任务卡。

【设计意图】计划先行是六步法的骨架；"预期证据"列强制学生动手前先定义"怎么算完成"，防实施阶段瞎试。

第 3 步：决策（20 分钟）

【教师活动】逐组 3 分钟计划答辩，重点拷问：①幂等实验如何设计才能证明"是去重生效，而不是第二次没提交成功"？②断路复盘你打算先查哪个接缝，为什么按这个顺序？

【学生活动】陈述计划 → 接受质询 → 修订计划表并签字（决策记录进入任务单）。

【教学方法】专业对话（Fachgespräch 雏形）。

【设计意图】决策 = "计划被批准"，模拟企业方案评审；在动手前暴露实验设计的逻辑漏洞，成本最低。

第 4 步：实施（90 分钟）

【教师活动】巡回指导，只在组卡死超过 5 分钟时给"方向性提示"（不给答案）；控制各子步骤节奏；收集各组典型错误投屏。

【学生活动】按计划表分工执行，记录员实时把命令、返回、耗时写入任务单证据栏。

【教学方法】任务驱动法 + 结对操作（驾驶员/导航员）。

【设计意图】四个子步骤 = 闭环正向走一遍（4.1→4.3）再逆向拆一遍（4.4），正反结合建立"链路感"。

4.1 直答与慢答对比：建立计时基线（20′）

```
cd dap-v2 && git switch edu/2026-spring
make gateway-forward        # 即 kubectl -n dap port-forward svc/dap-gateway 8000:8000
GW=http://localhost:8000

# ① 慢路径：题库未命中，走 LLM 全链路
time curl -s "$GW/api/query" -H "Authorization: Bearer $ANALYST_TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"question":"按品类统计上个月的 GMV","top_k":10}'
# 预期：route=llm，elapsed≈56000ms

# ② 直答路径：题库中已存在的存量 FAQ 题
time curl -s "$GW/api/query" -H "Authorization: Bearer $ANALYST_TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"question":"上月 GMV 的统计口径是什么"}'
# 预期：route=faq，elapsed≈420ms，响应含 "faq_source": true

kubectl -n dap logs deploy/insight-agent --tail=50 | grep -E "route=(faq|llm)"
```

记录员将两组（route / elapsed_ms / 是否含 SQL 执行）写入基线表——这是后面一切对比的参照系。

4.2 反馈闭环三组对照实验：三个安全设计（30′）

准备 `feedback.json`（corrected_sql 必须是真实修正，不是玩具语句）：

```
{
  "question_id": "q-edu-<学号>",
  "question": "按品类统计上个月的 GMV",
  "corrected_sql": "SELECT category, SUM(gmv - refund_amt) AS gmv FROM ads_gmv_daily WHERE dt =
date_trunc('month', current_date) - interval '1 month' GROUP BY category LIMIT 20",
  "reason": "原 SQL 漏扣退款金额"
}
# A. 正常提交 + 幂等重放（连续两次，间隔≤5s）
curl -s -X POST "$GW/api/feedback" -H "Authorization: Bearer $ANALYST_TOKEN" \
  -H 'Content-Type: application/json' -d @feedback.json
# → {"status":"ok","version":1}
```

```
curl -s -X POST "$GW/api/feedback" -H "Authorization: Bearer $ANALYST_TOKEN" \
  -H 'Content-Type: application/json' -d @feedback.json
# → {"status":"ok","dedup":true,"version":2}   ← 内容未变，行数不变

make crdb-sql    # 进入 CRDB SQL 终端（库 dap）
> SELECT question_id, tenant, version, updated_at FROM few_shots
  WHERE question_id = 'q-edu-<学号>';
-- 仍 1 行；version+1（幂等 UPSERT 的落库证据）

# B. 越权对照：viewer 提交同一反馈
curl -s -X POST "$GW/api/feedback" -H "Authorization: Bearer $VIEWER_TOKEN" \
  -H 'Content-Type: application/json' -d @feedback.json
# → 403 {"detail":"role analyst+ required"}   ← RBAC 证据

# C. 护栏预校验对照：三条坏 SQL 各提交一次（只改 corrected_sql 字段）
#   c1 "DROP TABLE edu_shadow.orders"          → 422 {"guardrail":"syntax"}   写操作拦截
#   c2 "SELECT * FROM ods_finance_orders LIMIT 10" → 422 {"guardrail":"tenant"} 跨租户表
#   c3 "SELECT * FROM ads_gmv_daily"            → 422 {"guardrail":"resource"} 缺强制 LIMIT
```

对照阅读 `feedback.py` 的调用顺序：先 `sql_check(corrected_sql, tenant)` 四层全过，才执行落库：

```
INSERT INTO few_shots (question_id, tenant, question, corrected_sql, version, ...)
VALUES ($1, $2, $3, $4, 1, ...)
ON CONFLICT (question_id, tenant) DO UPDATE
  SET corrected_sql = EXCLUDED.corrected_sql,
      version       = few_shots.version + 1,
      updated_at    = now();
```

4.3 增量索引与同题命中：闭环最后一公里（25′）

```
# ① 观察 worker 增量索引日志——重点核对写了哪些元数据字段
kubectl -n dap logs deploy/insight-worker --tail=100 | grep faq_index
# 预期：insert q-edu-<学号> fields=[vector, faq_question, faq_sql, tenant, enabled]

# ② 用 question 原文同题再问（不是用 question_id 问！）
time curl -s "$GW/api/query" -H "Authorization: Bearer $ANALYST_TOKEN" \
  -H 'Content-Type: application/json' \
  -d '{"question":"按品类统计上个月的GMV"}'
# 预期：route=faq，elapsed≈420ms —— 你 5 分钟前修正的 SQL 正在被复用

# ③ 命中计数（直答收益的量化口径）
curl -s "http://localhost:9090/api/v1/query" --data-urlencode \
  'query=faq_direct_hits_total{tenant="edu"}'
```

观察点（写入任务单）：L2 语义缓存命中后，服务层必须同时拿到 `is_faq` 与 `faq_sql` 两个透传字段——这正是断路事故的第 2 个断点。

4.4 "FAQ 断路"事故复盘：双层根因与断路测试（15′）

发放现象卡片（真实事故脱敏）：

few_shots 表有行、Milvus 实体数 +1，但同题永远 56s；全程无 error 日志，监控全绿，两周无人发现。

小组按五问清单逐接缝验证并勾选：

#	接缝	当时的验证命令	事故真相
1	存储在吗	`SELECT * FROM few_shots ...`	✔ 在
2	索引进了吗	Milvus `num_entities` +1	✔ 进了
3	命中了吗	检索日志 score ≥ 阈值	✔ 命中
4	元数据全吗	取回 entity 的	**✘ R1：字段为空（worker**

		faq_question/faq_sql	**写入漏带元数据）**
5	透传到了吗	服务层日志是否收到 is_faq/faq_sql	**✘ R2：L2 命中结构透传时 metadata 被丢弃**

研讨两问（写入任务单）：①为什么单修任何一个 bug 都不够？（R1、R2 任一存在，健康路径恒不命中。）②为什么两周才发现？（两层叠加后系统静默降级到 LLM——结果仍正确、只是慢，不报错、不告警，这是最危险的断路形态：伪健康。）

随后翻看 tests/faq/test_faq_regression.py 的 57 条回归（覆盖五环 × 正反例 × 耗时断言 <2s），每组设计"第 58 条测试"（Given/When/Then 三段式，企业站可真正实现）。

第 5 步：检查（40 分钟）

【教师活动】发布 10 项检查清单，逐组核对证据；未达成项如实标注"未验证/未完成"（诚实文化：如实标注不扣诚信分，虚报按 0 分）。

【学生活动】自查 + 组间互查，重点核对：

1. 基线表含 route=llm 与 route=faq 两行计时数据；
2. 幂等：两次响应 JSON ＋ 前后 SQL 查询结果对比；
3. 403 × 1、422 × 3 的返回体原文；
4. worker 日志出现 4 个元数据字段名；
5. 同题命中证据（<2s ＋ route=faq ＋ few_shots 行）；
6. 五问清单 5 行全部勾选且正确标注 R1/R2 两个断点；
7. 第 58 条测试三段式完整；
8. 任务单"未完成清单"栏如实填写。

【教学方法】核对表法 + 同伴互评。

【设计意图】"无产物不声称完成"——每项检查对应可复查的证据，训练证据意识与诚实自评（HSA）。

第 6 步：评价＋学生讲堂（30 分钟）

【教师活动】主持 15′学生讲堂（抽 3 组各 4′）并点评；10′按评分表评分；5′布置企业站任务与课后费曼任务。

【学生活动】讲堂主题抽签二选一：①"我如何 15 分钟定位一个监控全绿的故障"；②"为什么 420ms 里最贵的不是计算，是那曾经修对过一次的 SQL"。听众按互评量表打分（清晰度 / 证据 / 类比各 5 分）。

【教学方法】费曼法 + 同伴互评量表。

【设计意图】把分析过程讲明白才算真正掌握；主题②为课后费曼任务"知识引擎是天花板"埋伏笔。

七、企业站实践任务卡（2 学时）

项目	内容
任务目标	在 dap-v2 课程分支完成"提交一条真实反馈并验证同题直答命中"的完整闭环 PR
场地/协作	企业实训站；2 人结对（驾驶员/导航员 20′轮换），企业导师巡回评审
步骤	① 从本学期评测集挑 1 道错题（保留自己当时的错误 SQL 证据）→ ② 人工写出正确 SQL，本地先过 `sql_check` 四层护栏 → ③ 以 analyst 提交 feedback 并做幂等重放 → ④ 等待 worker 增量索引（≤60s，看日志）→ ⑤ 同题直答计时（截取 route=faq 日志行）→ ⑥ 补一组恶意 SQL 拦截对照 → ⑦ 撰写案例文档并提 PR
交付物	PR 含：`docs/07-项目资源/faq-cases/<学号>.md`（问题/原 SQL/修正 SQL/护栏对照/前后耗时对比表）＋关键日志与响应证据粘贴
评审标准	① 直答命中证据链完整（few_shots 行＋worker 日志＋<2s＋route=faq）② 护栏拦截至少 1 组 4xx 对照 ③ 文档三查（规范/可复现/结论诚实，未验证项如实标注）④ PR 描述含一段"如果断路发生在我这题，会断在哪一环"的分析
双导师签栏	企业导师（签名/日期）：_____ 校内导师（签名/日期）：_____

八、评价设计（任务单评分表，满分 100）

维度	权重	评价标准
资讯与计划	15	五环链路图完整、每环失败形态标注清楚；计划表含"预期证据"列
实施完成度	35	三组对照实验全部复现（15）；同题命中证据齐全（10）；worker 元数据观察记录（10）
断路复盘质量	20	五问清单验证命令正确（10）；双层根因及"单修不够"论证成立（10）
检查与证据	15	证据齐全可复查；未完成项如实标注"未验证"而非空着
学生讲堂表达	15	外行能听懂（类比准确）、有计时/日志证据支撑、4 分钟内讲完

等级：≥90 优秀 / 75–89 良好 / 60–74 合格 / <60 重做（限补一次）。

九、课后费曼任务

- **讲给谁**：非技术专业的同学或家人（对方不懂 SQL 与缓存）。
- **讲什么**："知识引擎是 Agent 效果天花板"——用两次提问（56 秒与 0.4 秒）和断路事故当故事：模型再强，喂错知识照样答错；修对一次的 SQL 若不存下来，下次还会错一遍。
- **自评三问**：①我不看图能讲出反馈闭环五环吗？②对方能用自己的话复述"为什么要存下来"吗？③我讲的断路事故，对方能说出"坏在哪一环"吗？——任一问答"否"，重讲一遍并录音自查。

十、教学反思栏（授课后 48 小时内填写）

反思维度	现象记录	原因分析	改进措施
目标达成度（5 条目标逐条）			
难点突破效果（静默故障定位）			
时间分配偏差（哪一步超时）			
学生典型错误与误解			
企业站衔接是否顺畅			
下一讲迭代事项（衔接 L10 缓存层）			